

Análisis y resolución del problema de exact cover mediante backtracking

Santiago Emmanuel Jimenez Perez

21 de mayo de 2026

1. Resumen

En este documento exploramos cómo analizar y resolver computacionalmente el problema de Exact Cover. Primero, construimos una solución inicial usando búsqueda exhaustiva (Backtracking) y, posteriormente, demostramos cómo optimizar este algoritmo aplicando principios matemáticos al árbol de decisiones. Esto nos permite reducir drásticamente los tiempos de procesamiento en la práctica

2. Introducción

El problema de (*Exact Cover*) es un problema de decisión clásico en la teoría de la complejidad computacional. Dado un conjunto universo finito y una familia de subconjuntos de este universo, el objetivo es determinar si existe una subcolección que sea una partición exacta del conjunto original.

Formalmente, de acuerdo con Kreher y Stinson [1], el problema se define de la siguiente manera: Sea $\mathcal{R} = \{0, 1, \dots, n-1\}$ un conjunto universo y sea $\mathcal{S} = \{S_0, S_1, \dots, S_{m-1}\}$ una colección de subconjuntos de $\mathcal{R}$.

El objetivo es encontrar si existe una subcolección $\mathcal{S}' \subseteq \mathcal{S}$ tal que:

$$\bigcup_{S_i \in \mathcal{S}'} S_i = \mathcal{R} \quad (1)$$

$$\forall S_i, S_j \in \mathcal{S}', i \neq j \implies S_i \cap S_j = \emptyset \quad (2)$$

3. Marco teórico

3.1. Complejidad computacional

El *Exact Cover* es un problema de decisión NP-completo, lo que significa que el tiempo necesario para resolverlo aumenta de forma rápida al crecer el problema. Al no existir un algoritmo de tiempo polinomial conocido, la optimización estructural del espacio de búsqueda es indispensable.

3.2. Conceptos fundamentales

- **Backtracking:** Es una técnica de búsqueda que construye la solución paso a paso. Si el algoritmo llega a un punto donde no puede avanzar, retrocede para intentar otro camino.
- **Exact 3-cover:** Es una variante específica donde cada subconjunto disponible tiene exactamente 3 elementos. Este problema es extremadamente difícil porque las restricciones de intersección son muy estrictas, lo que lo convierte en una prueba ideal para medir la eficiencia algorítmica.

4. Explicación del algoritmo y comparativa de eficiencia

4.1. El algoritmo base (programa 1): fuerza bruta

Como primera idea se plantea buscar la solución mediante fuerza bruta. El Programa 1 recorre linealmente la colección de subconjuntos e intenta añadirlos a la solución paso a paso. Si detecta que un subconjunto choca con otro, retrocede. El problema es que este método "no piensa"; explora miles de combinaciones que desde el inicio no tenían posibilidad de funcionar, lo que consume excesiva memoria y tiempo.

4.2. El algoritmo optimizado (programa 2): poda mediante intersección

Para resolver esto, el Programa 2 utiliza una lógica más inteligente. En lugar de probar todo a ciegas, el algoritmo hace dos cosas antes de avanzar:

1. **Filtro previo:** Solo considera los conjuntos que realmente pueden cubrir los elementos que faltan.
2. **Poda de intersección:** Al seleccionar un conjunto, descarta automáticamente todos los demás que tengan elementos en común. Así, el árbol de búsqueda se vuelve mucho más pequeño y rápido.

4.3. Pseudocódigo

Para comprender la diferencia estructural de lo explicado anteriormente, presentamos el pseudocódigo formal de los algoritmos implementados:

Algoritmo 1: fuerza bruta

1. **Input:** Universo U , Colección S
2. **Si** $U = \emptyset$, retorna *solución*.
3. **Para cada** $s \in S$:
4. **Si** $s \cap \text{cubiertos} = \emptyset$:
5. Añadir s a *solución*.
6. **Recursión**.
7. **Backtracking**.

algoritmo 2: poda algebraica

1. **Input:** Universo U , Colección S
2. **Si** $U = \emptyset$, retorna *solución*.
3. **Filtrar** $S' = \{s \in S \mid s \cap S_{\text{actual}} = \emptyset\}$
4. **Para cada** $s \in S'$:
5. Añadir s a *solución*.
6. **Recursión**.
7. **Backtracking**.

5. Análisis comparativo y metodología de pruebas

Para evaluar rigurosamente el impacto de la poda por conjuntos disjuntos, se sometieron ambos algoritmos a varias pruebas progresivas.

5.1. Definición de las métricas

- **U (Universo):** Representa el tamaño del conjunto de elementos que deben ser cubiertos. Es la complejidad del rompecabezas.^a resolver.
- **S (Familia de subconjuntos):** Indica la cantidad total de subconjuntos disponibles en un inicio. Un número mayor significa más opciones que debe evaluar el algoritmo.
- **S' (Solución):** Cardinalidad de la subcolección que cumple las condiciones de Exact Cover. Si el valor es $\emptyset$, indica que el algoritmo recorrió la totalidad del espacio de búsqueda sin hallar una solución.
- **Aceleración (T_1/T_2):** Cociente que mide cuántas veces más rápido fue el Programa 2 respecto al Programa 1.

5.2. Escenarios de Prueba

1. **Caso Trivial:** caso pequeño para validar el funcionamiento de ambos programas.
2. **Prueba de Estrés:** Base de datos moderado diseñado para medir la capacidad de respuesta ante choques constantes.
3. **Base de datos “Envenenada”:** Prueba en la que se analizan 30,000 conjuntos basura, obligando al algoritmo a descartar opciones masivamente.
4. **Prueba masiva:** Base de datos basada en el problema *Exact 3-Cover*, forzando a los algoritmos a una búsqueda profunda en la que exploración de aproximadamente 135,000 nodos.

Escenario	U	S	S'	Prog. 1	Prog. 2	Aceleración
Caso Trivial	7	6	3	0.032 ms	0.023 ms	1.4x
Prueba de Estrés	30	406	6	2.20 ms	0.40 ms	5.5x
Dataset “Envenenado”	12	30,066	$\emptyset$	57.8 ms	13.0 ms	4.4x
Prueba masiva	14	10,056	$\emptyset$	121.09 s	67.28 s	1.8x

Cuadro 1: Comparativa de rendimiento .

Resultados: Como se observa en el Cuadro1, en situaciones con una profundidad de árbol moderada, El programa optimizado alcanza aceleraciones de hasta 5.5 veces. En la *Prueba masiva*, el Programa 2 logra evadir el colapso, reduciendo el tiempo de ejecución en aproximadamente 54 segundos. Esto demuestra que la condición de poda $S_i \cap S_j = \emptyset$ es importante para que el algoritmo sea viable frente a casos complejos.

6. Conclusión

Los cambios del algoritmo de fuerza bruta al optimizado demuestra que la condición matemática $S_i \cap S_j = \emptyset$ no es solo un detalle de implementación, sino una pieza fundamental. Aplicar conceptos de teoría de conjuntos para purgar el árbol de estado es crítico para mantener la viabilidad del algoritmo frente a casos complejos, validando la importancia de modelar matemáticamente las restricciones antes de programarlas.

Referencias

- [1] Kreher, D. L., & Stinson, D. R. (1999). *Combinatorial Algorithms: Generation, Enumeration, and Search*. CRC Press. (pp. 118-121).